

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	K HARSHIKA	Reg. No.:	192473020
Section / Batch:	CSE-DS	Date:	26/08/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct

I K HARSHIKA(192473020)_____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: HARSHIKAREDDY_____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SAT Problem (NP-Complete / Boolean Constraint Problem)

Introduction

The 3-SAT (3-Satisfiability) problem is an important problem in computational complexity and Boolean logic. In this problem, a Boolean formula is given in Conjunctive Normal Form (CNF), where each clause contains exactly three literals. The objective is to determine whether there exists an assignment of True or False values to the Boolean variables such that the entire formula becomes True.

A literal can be either a variable or its negation. For example, if x is a Boolean variable, then x and $\neg x$ are literals.

A general 3-SAT formula is represented as:

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4) \wedge \cdots (x_1 \vee \neg x_2 \vee x_3) \vee (\neg x_1 \vee x_2 \vee x_4) \vee \cdots$$

The formula is satisfiable if at least one assignment makes every clause True.

Example Problem

Consider the following Boolean formula:

$$F = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \neg x_3) \\ F = (x_1 \vee \neg x_2 \vee x_3) \vee (\neg x_1 \vee x_2 \vee x_3) \vee (x_1 \vee x_2 \vee \neg x_3)$$

Each clause contains three literals.

The constraints are:

➤ **Clause 1:** At least one of x_1 , $\neg x_2$, or x_3 must be True.

➤ **Clause 2:** At least one of $\neg x_1$, x_2 , or x_3 must be True.

➤ **Clause 3:** At least one of x_1 , x_2 , or $\neg x_3$ must be True.

•

Since the clauses are connected using the AND operator, **every clause must be satisfied** for the complete formula to be satisfied.

Variable Assignment

Each Boolean variable has two possible values:

$$x_i \in \{\text{True}, \text{False}\} \quad x_i \in \{\text{True}, \text{False}\}$$

For n variables, there are:

$$2^n$$

possible assignments.

For example, with three variables:

$x_1 \ x_2 \ x_3$

F F F

F F T

F T F

F T T

T F F

T F T

T T F

T T T

The objective is to find at least one assignment that satisfies all clauses.

Backtracking-Based Solution

A backtracking algorithm assigns values to variables one by one.

For every variable, there are two choices:

➤ Assign True

➤ Assign False

After each assignment, the algorithm checks whether any clause has become permanently unsatisfied. If a clause is already unsatisfied, there is no need to continue exploring that branch. This is called **pruning**.

The basic steps are:

➤ Select an unassigned variable.

➤ Assign it the value True.

- Check whether the partial assignment violates any clause.
- If no violation occurs, continue with the next variable.
- If a violation occurs, backtrack.
- Repeat the process by assigning False.
- If all variables are assigned and every clause is satisfied, return satisfiable.

Backtracking Algorithm

SAT(variable_index):

```

    if all variables are assigned:
        if all clauses are satisfied:
            return True
        else:
            return False

    Assign current variable = True

    if no clause is unsatisfied:
        if SAT(next variable) is True:
            return True

    Assign current variable = False

    if no clause is unsatisfied:
        if SAT(next variable) is True:
            return True

    Remove assignment
    return False

```

Pruning Technique

Pruning improves the backtracking process by eliminating unnecessary search branches.

Consider a clause:

$$(x_1 \vee x_2 \vee x_3) \quad (x_1 \text{ \texttt{\textbackslash}lor } x_2 \text{ \texttt{\textbackslash}lor } x_3)$$

If a partial assignment produces:

$x_1 = \text{False}, x_2 = \text{False}, x_3 = \text{False}$
 $x_1 = \text{False}, \quad x_2 = \text{False}, \quad x_3 = \text{False}$

then the clause becomes:

$\text{False} \vee \text{False} \vee \text{False} = \text{False}$
 $\text{False} \vee \text{False} \vee \text{False} = \text{False}$

Since clauses are connected using AND, the entire formula can never become True under this branch. Therefore, the algorithm immediately stops exploring this assignment.

This avoids checking the remaining assignments and improves practical performance.

Interaction Between Constraints

The main difficulty in 3-SAT is that the same variable can appear in multiple clauses.

For example:

$(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_4 \vee x_5)$
 $(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_4 \vee x_5)$

The assignment of x_1 affects both clauses.

If:

$x_1 = \text{True}$
 $x_1 = \text{True}$

the first clause is immediately satisfied, but the literal $\neg x_1$ becomes False in the second clause.

Thus, decisions that satisfy one clause may create restrictions on another clause. The backtracking algorithm handles these interactions by continuously checking all affected clauses.

Correctness

The backtracking algorithm is correct because it systematically explores all possible Boolean assignments unless a branch is pruned.

A branch is pruned only when at least one clause is already completely False. Since a false clause cannot become True by assigning additional variables, no valid solution can exist in that branch.

Hence, the algorithm correctly determines whether the given 3-SAT formula is satisfiable.

Complexity Analysis

For n Boolean variables, each variable has two possible assignments.

Therefore, the maximum number of assignments is:

2^n

Time Complexity

In the worst case:

$$O(2^n \cdot m)$$

The factor m represents checking the clauses for each assignment.

Space Complexity

The recursive backtracking stack requires:

$$O(n)$$

The formula and assignments also require additional storage. Therefore, the basic recursion depth is:

$$O(n)$$

NP-Complete Nature of 3-SAT

The 3-SAT problem belongs to the class **NP-Complete** problems.

1. 3-SAT belongs to NP

If a satisfying assignment is provided, we can verify whether every clause is True in polynomial time.

For each clause, only three literals need to be checked.

Therefore, verification is efficient.

2. 3-SAT is NP-Hard

The 3-SAT problem is NP-Hard because other problems in NP can be transformed or reduced to it in polynomial time.

Therefore:

3-SAT is NP-Complete

This means that 3-SAT is one of the fundamental problems used to study computational complexity and difficult combinatorial search problems.

CODE

```
# =====  
  
# Q16: 3-SAT Problem using Backtracking and Pruning  
  
# =====  
  
# Each clause contains 3 literals.  
  
# Positive number = variable
```

```
# Negative number = NOT variable
```

```
#
```

```
# Example:
```

```
# 1 means x1
```

```
# -2 means NOT x2
```

```
#
```

```
# Formula:
```

```
# (x1 OR NOT x2 OR x3)
```

```
# AND
```

```
# (NOT x1 OR x2 OR x3)
```

```
# AND
```

```
# (x1 OR x2 OR NOT x3)
```

```
clauses = [
```

```
    [1, -2, 3],
```

```
    [-1, 2, 3],
```

```
    [1, 2, -3]
```

```
]
```

```
num_variables = 3
```

```
# -----
```

```
# Check whether a literal is True or False
```

```
# -----
```

```
def literal_value(literal, assignment):
```

```
    variable = abs(literal)
```

```
# Variable not assigned yet
```

```
if variable not in assignment:
```

```
    return None
```

```
value = assignment[variable]
```

```
# Positive literal: x
```

```
if literal > 0:
```

```
    return value
```

```
# Negative literal: NOT x
```

```
return not value
```

```
# -----
```

```
# Check if any clause is already unsatisfied
```

```
# Used for pruning
```

```
# -----
```

```
def has_conflict(clauses, assignment):
```

```
    for clause in clauses:
```

```
        clause_satisfied = False
```

```
        unassigned_exists = False
```

```
        for literal in clause:
```

```
            value = literal_value(literal, assignment)
```

```
if value is True:

    clause_satisfied = True

    break
```

```
elif value is None:

    unassigned_exists = True
```

```
# Clause is completely false

if not clause_satisfied and not unassigned_exists:

    return True
```

```
return False
```

```
# -----
```

```
# Check whether all clauses are satisfied
```

```
# -----
```

```
def is_satisfied(clauses, assignment):
```

```
    for clause in clauses:
```

```
        clause_satisfied = False
```

```
        for literal in clause:
```

```
            if literal_value(literal, assignment) is True:
```

```
                clause_satisfied = True
```

```
                break
```

```

        if not clause_satisfied:

            return False

    return True

# -----
# Backtracking algorithm
# -----

def solve_3sat(clauses, num_variables, assignment=None):

    if assignment is None:

        assignment = {}

    # Pruning: stop if a clause is already unsatisfied

    if has_conflict(clauses, assignment):

        return None

    # If all variables are assigned, check complete formula

    if len(assignment) == num_variables:

        if is_satisfied(clauses, assignment):

            return assignment.copy()

        return None

    # Find next unassigned variable

    for variable in range(1, num_variables + 1):

        if variable not in assignment:

            break

```

```

# Try True

assignment[variable] = True

result = solve_3sat(clauses, num_variables, assignment)

if result is not None:

    return result

# Try False

assignment[variable] = False

result = solve_3sat(clauses, num_variables, assignment)

if result is not None:

    return result

# Backtrack

del assignment[variable]

return None


# =====

# RUN THE PROGRAM

# =====


print("=" * 55)

print("3-SAT PROBLEM USING BACKTRACKING AND PRUNING")

print("=" * 55)

```

```
print("\nBoolean Formula:")

for i, clause in enumerate(clauses, start=1):
    print(f"Clause {i}: {clause}")

print("\nSolving...")

solution = solve_3sat(clauses, num_variables)

print("\n" + "=" * 55)

if solution:
    print("RESULT: SATISFIABLE")
    print("\nSatisfying Assignment:")

    for variable in sorted(solution):
        print(f"x{variable} = {solution[variable]}")

else:
    print("RESULT: UNSATISFIABLE")

print("\n" + "=" * 55)
```

OUTPUT

```
... =====
3-SAT PROBLEM USING BACKTRACKING AND PRUNING
=====

Boolean Formula:
Clause 1: [1, -2, 3]
Clause 2: [-1, 2, 3]
Clause 3: [1, 2, -3]

Solving...

=====
RESULT: SATISFIABLE

Satisfying Assignment:
x1 = True
x2 = True
x3 = True
=====
```

Conclusion

The 3-SAT problem determines whether a Boolean formula containing clauses of three literals can be satisfied by assigning True or False values to its variables. The backtracking approach systematically explores possible assignments, while pruning removes branches that already contain an unsatisfied clause. This improves practical efficiency, although the worst-case time complexity remains exponential.

The problem demonstrates how constraints interact because a single variable may affect multiple clauses. The correctness of the algorithm follows from the fact that all possible assignments are considered unless they are guaranteed to fail. Since a proposed solution can be verified in polynomial time and the problem is NP-Hard, 3-SAT is classified as an NP-Complete problem.

Focus Area: This problem focuses on solving Boolean constraint satisfaction problems using backtracking and pruning techniques. It also demonstrates the concept of NP-Completeness and the challenges involved in exponential search problems.